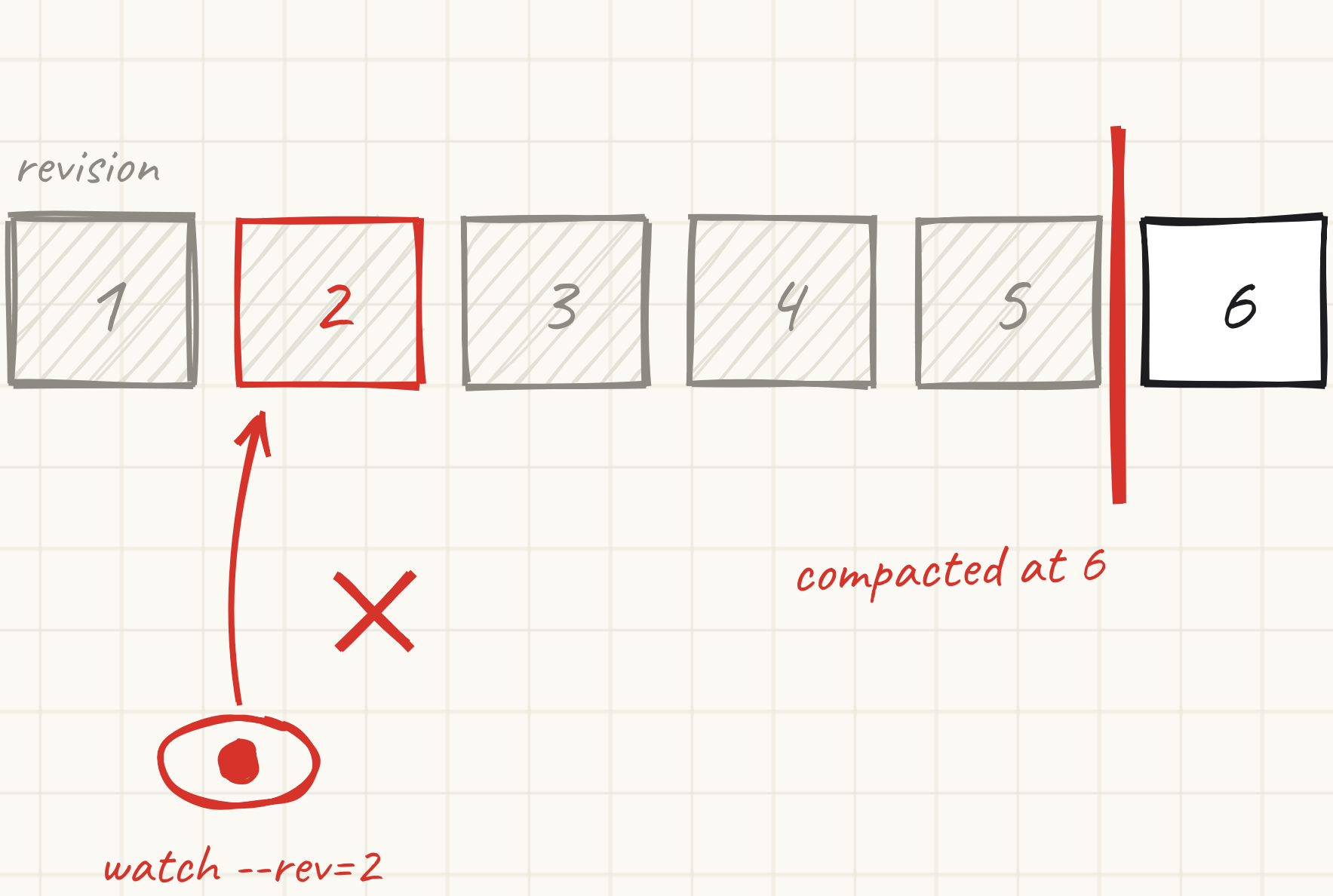
Your watch died. History is gone.

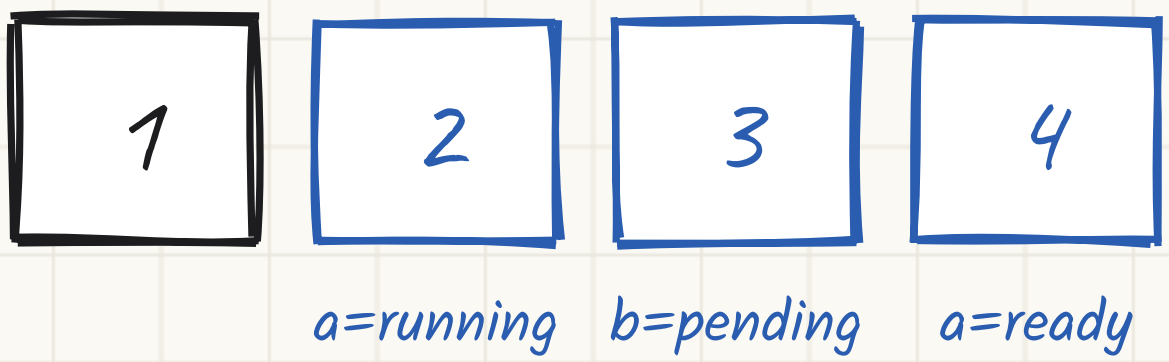
It is not a flaky cluster. etcd threw that history away on purpose.



Every write gets a number.

etcd keeps one counter for the whole cluster. Each put bumps it by one.

revision

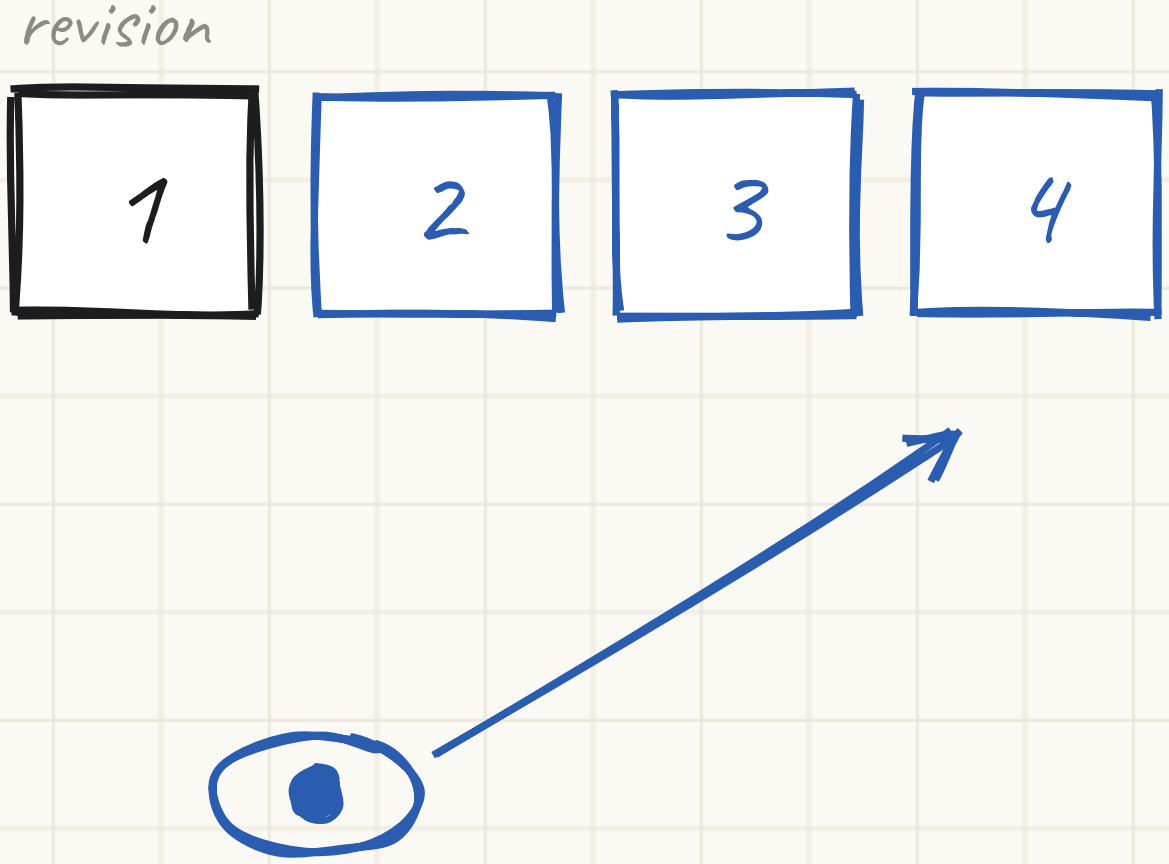


put a, put b, update a -> revision 4

So you can rewind.

A watch can start in the past.

Ask for revision 2 and etcd replays every change since. That is how a controller catches up after a blip.

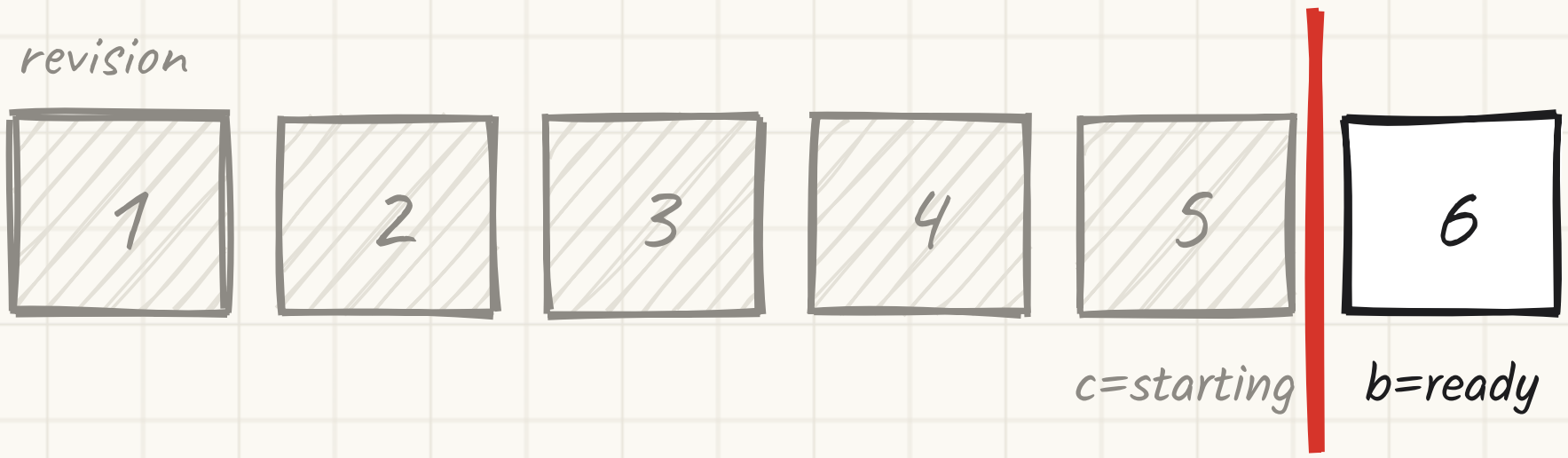


events_seen=3✓

But history costs disk.

Compaction throws old history away.

Two more writes, then `compact 6`.
Everything before 6 is gone for good.

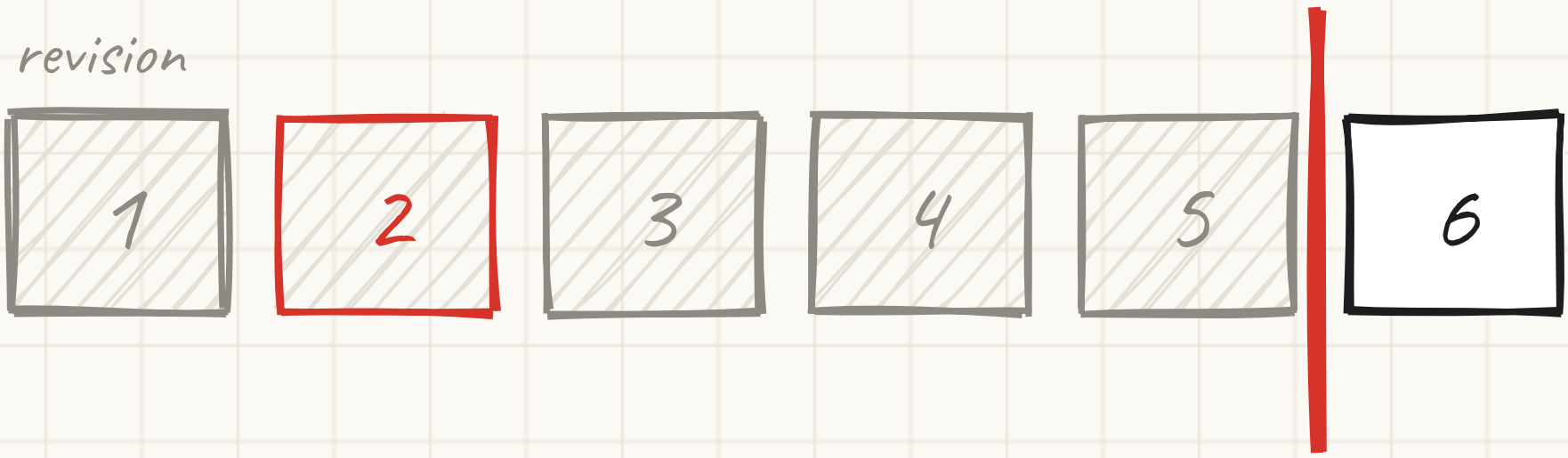



```
compacted revision 6
```

Now your controller reconnects.

Wrong: resume from 2 again.

The server cancels the watch. Real output:



compacted at 6



watch --rev=2

```
watch was canceled (etcdserver: mvcc:
required revision has been compacted)
```

Retry it? Same answer.

Retrying loops forever.

Revision 2 will never come back. Even `get --rev=2` fails. Go clients see this as *ErrCompacted*.



a history gap, not a network blip

So what is right?

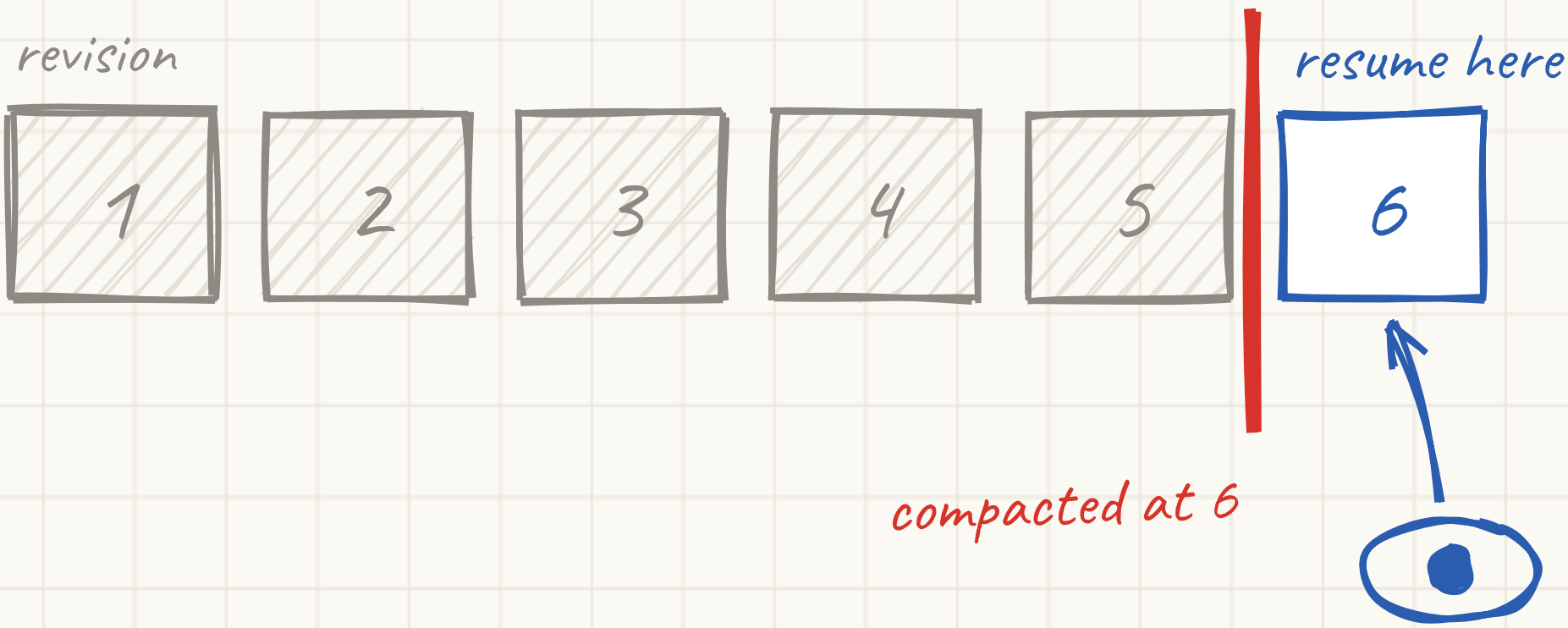
Right: re-list, then watch.

Read what exists now. Then watch from a revision etcd still has.

1. `get --prefix /demo/pod/`

   `a=ready b=ready c=starting`

2. `watch --rev=6`



One catch.

The gap is not replayed.

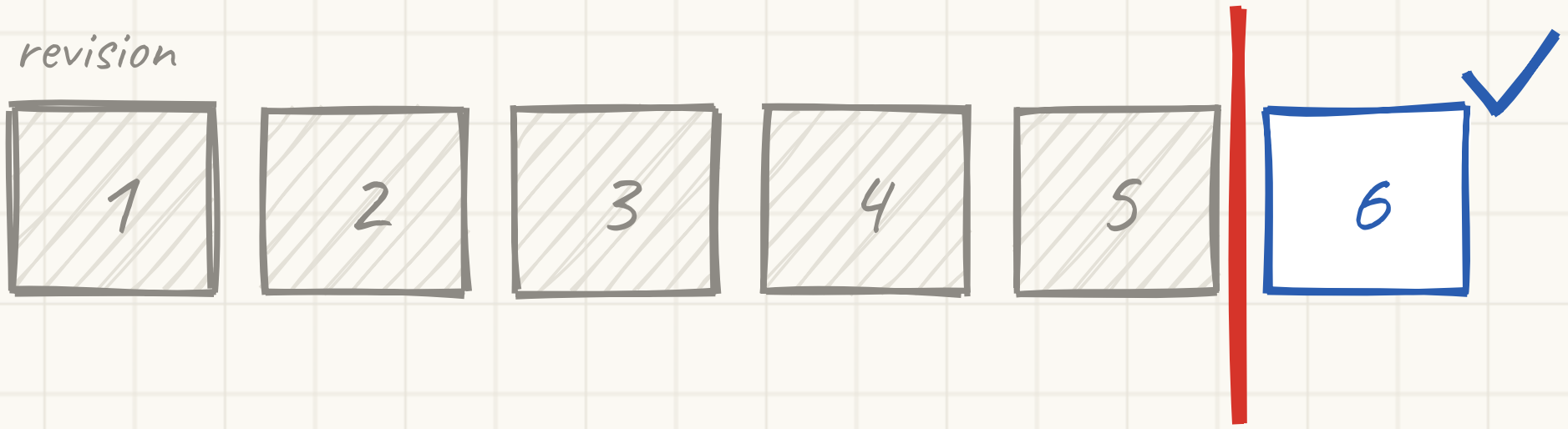
Changes between your last revision and the wall are gone. The list is your truth now. Auto-compaction moves the wall too.



So, the whole drawing.

So, the whole drawing.

Revisions tick up. Compaction drops the past. Hit the wall? Re-list, then watch from what is left.



How does your controller handle ErrCompacted today?